Towards AI · Following

The Orchestration Tax: Why Multi-Agent Systems Get Expensive

How context propagation, supervisor loops, tool calls, memory, and observability quietly drive up the cost of production agentic systems.

12 min read · 5 hours ago

Deepanshu Gupta Follow

Multi-agent AI systems are quickly becoming a default pattern for building advanced LLM applications. Instead of relying on one model loop to plan, reason, retrieve, call tools, validate outputs, remember context, and produce the final answer, teams split the work across several specialised components.

A planner decomposes the task. A supervisor routes work. Specialist agents handle narrow responsibilities. Tool agents interact with APIs, search systems, databases, or internal services. A verifier checks the output. A memory layer provides continuity. An observability system records what happened.

This looks like sensible software architecture. Responsibilities are separated. Agents can specialise. Workflows become easier to monitor. Complex tasks can be broken down into smaller steps.

But once these systems move from demos to production, a second reality appears:

> *multi-agent systems are expensive by construction.*

That does not mean they are bad. It means they need to be designed with cost as a first-class architectural concern.

A single user request may look simple from the outside. Internally, it may trigger a planning call, several specialist calls, multiple tool calls, memory retrieval, validation, retries, tracing, and sometimes evaluation. What the user sees as one response is often the output of a small distributed inference system.

The cost problem is not simply that LLMs charge per token. The deeper issue is that agentic systems multiply tokens, calls, state, and infrastructure activity. Every agent boundary creates another chance to replay context. Every supervisor decision adds latency and model usage. Every tool call creates a loop of selection, execution, interpretation, and possible retry. Every trace span must be stored, queried, and sometimes replayed.

This is the "***orchestration tax***".

The important question is not only:

> ***Which model is cheapest?***

The more useful question is:

> ***How much useful progress does this system make for every model call, token, tool execution, and second of latency it consumes?***

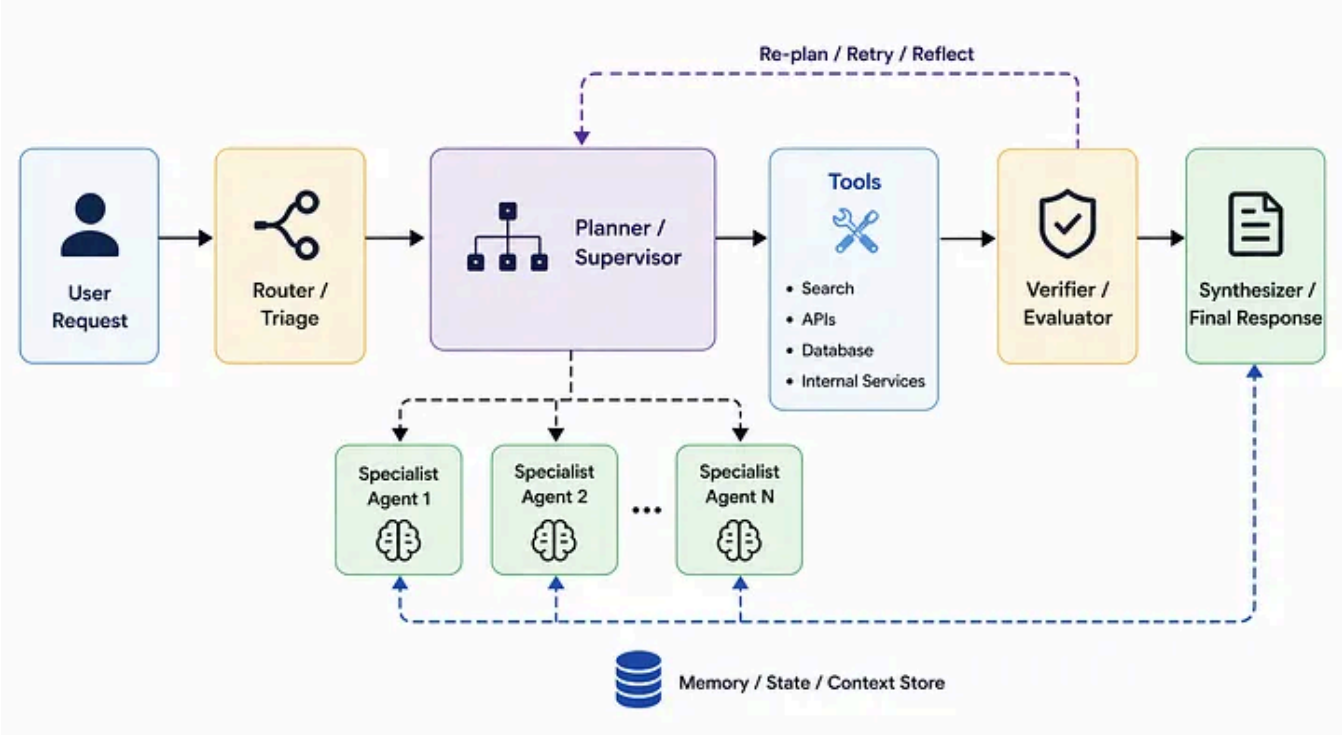


Figure 1. A Multi-agent System Architecture

Multi-agent systems are distributed inference workflows

A traditional LLM application often has a simple execution shape. A user sends a request, the system builds a prompt, the model generates a response, and the answer is returned. Tools, retrieval, and memory may exist, but they usually sit around one model loop.

A multi-agent application changes that shape.

The system may first ask a router to classify the request. A planner may then decompose it. A supervisor may assign subtasks to agents. One agent may search documents. Another may query a database. Another may summarise findings. A verifier may check whether the answer is grounded. A final synthesiser may write the user-facing response.

The user still receives one answer, but the system has executed many reasoning and orchestration steps.

```
User request
↓
Triage / router
↓
Supervisor or planner
```

```
↓
Specialist agents
↓
Tools and memory
↓
Verifier
↓
Final response
↓
Trace and evaluation logs
```

Each step can be valuable. The problem is that each step has a cost. If the system does not control when these steps are actually needed, the architecture becomes expensive before traffic even scales.

A better way to think about multi-agent cost is to stop thinking in terms of a single prompt and start thinking in terms of execution paths. Each request travels through a graph of possible model calls, tools, memory lookups, retries, checks, and logs. Some paths are short and cheap. Others are long and expensive.

The goal is not to make every path minimal. Some tasks genuinely require deeper reasoning. The goal is to ensure expensive paths are used only when the task justifies them.

A low-risk formatting task should not activate the same workflow as a high-risk multi-document analysis. A simple classification should not go through a frontier model planner. A verifier should not run when deterministic validation is enough. Retrieval should not happen when the answer is already available in the current state.

> *Cost-aware design starts by recognising that an agentic system is not just a group of prompts. It is a runtime system.*

Why cheaper models alone do not fix agentic cost

When teams first notice rising LLM costs, the instinct is often to move work to cheaper models. This is sensible, but incomplete.

Smaller models are essential in production. They are often sufficient for classification, extraction, routing, formatting, summarisation, schema repair, and many deterministic-adjacent tasks. Expensive reasoning models should be reserved for harder decisions, ambiguous planning, complex synthesis, or high-risk judgement.

But model substitution does not fix architectural waste.

A cheap model that receives a bloated prompt can still be expensive. A small worker model called ten times because the planner keeps retrying can still drive the cost up. A low-cost model that repeatedly invokes search, database calls, and validators may not be cheap at the workflow level. A system that sends full conversation history to every agent will repeatedly pay for irrelevant context regardless of model choice.

This is the core distinction:

> *Model optimisation reduces the unit price of inference. Architecture optimisation reduces the amount of inference required.*

Both matter, but architecture determines the shape of the bill.

The main place cost explodes: context propagation

The most common source of cost growth in multi-agent systems is context propagation.

Context propagation is the movement of information from one step of the workflow to another. In a good system, each agent receives the information it needs to perform its task. In a poor system, each agent receives everything because that is easier to implement.

The second approach is convenient during prototyping. It reduces the risk that an agent is missing information. It also makes debugging easier because every step has access to the full state.

But convenience becomes expensive.

Imagine a workflow where the first agent receives the user request, the second receives the user request plus the first agent's output, the third receives the user request plus the first and second outputs, and so on. The context grows after every step. If multiple agents are activated at each stage, the same accumulated state is processed repeatedly.

```
Step 1:
User request

Step 2:
User request
+ planner output

Step 3:
User request
+ planner output
+ tool results

Step 4:
User request
+ planner output
+ tool results
+ worker output

Step 5:
User request
+ planner output
+ tool results
+ worker output
+ verifier notes
```

The later calls are more expensive than the earlier calls, even if the underlying task has not become harder. The system is paying for historical baggage.

This is especially dangerous in supervisor systems, hierarchical agents, and handoff architectures. If each agent inherits the full parent state, the workflow can become expensive quickly. In conversational systems, the problem grows further because user history accumulates across turns.

A better design is to compile context deliberately.

Instead of passing the whole state, the system builds a focused payload for the next agent. That payload should describe the task, the relevant facts, the required

evidence, the constraints, and the expected output format.

For example, a verifier does not need the entire conversation. It needs the final answer, the evidence used to produce it, and the rubric for checking it.

```
{
"task": "Check whether the answer is supported by the evidence",
"answer": "The generated response text",
"evidence": [
"Relevant source snippet 1",
"Relevant source snippet 2"
],
"criteria": [
"factual consistency",
"unsupported claims",
"missing caveats"
],
"required_output": "pass | fail | escalate"
}
```

This is not only cheaper. It is often more reliable. The agent has less irrelevant information to process, and the task boundary is clearer.

Figure 2. Token Growth Across Workflow Steps

The practical lesson is simple:

> *Context should be engineered, not inherited automatically.*

A production-grade multi-agent system should treat context assembly as a core runtime function. It should decide what to include, what to summarise, what to retrieve, what to omit, and what to store externally.

This is where many agent systems win or lose economically.

Supervisor architectures add control, but also overhead

Supervisor architectures are popular because they make multi-agent systems easier to control.

A supervisor can inspect the user request, choose the right specialist, manage tool access, coordinate intermediate outputs, enforce policies, decide whether verification is required, and produce the final response. For complex workflows, this is valuable.

But the supervisor is also an additional reasoning layer.

Every supervisor step can add a model call. The system may call the supervisor to decide what to do, again after a worker responds, again after a tool failure, and again before final synthesis. If the supervisor is a strong model, that fixed overhead can become significant.

This is not always a problem. In regulated or high-risk workflows, supervisor cost may be justified because it reduces mistakes. In long-running tasks, a planner may prevent wasted downstream work. In multi-domain systems, a supervisor may avoid giving every agent every tool and every piece of context.

The problem appears when a supervisor is used where simpler control would work.

A customer support ticket that only needs category classification may not need a supervisor. A JSON validation task does not need an LLM manager. A formatting task does not need a planning loop. A simple extraction job may only need one structured-output call.

Supervisor overhead should be justified by the value it creates.

> *Add a supervisor when coordination is necessary, not by default.*

A mature system may combine both approaches. Low-risk requests go through deterministic routing or a small model. Higher-risk requests activate the supervisor. Complex requests activate the planner, specialists, tools, and verifier.

This turns the supervisor from a universal wrapper into a conditional control mechanism.

Tool calls are cost multipliers

Tool calling is another major source of hidden cost.

At first, a tool call appears cheap. The model emits a structured function call, an external system returns a result, and the agent continues. But the full cycle is more expensive than it looks.

The model must decide whether a tool is needed. It must choose the correct tool and arguments. The external tool must execute. The result must be returned to the model. The model must interpret the result. If the result is incomplete, malformed, or unexpected, the system may retry, repair, or call another tool.

A single tool interaction can therefore involve multiple model and infrastructure steps.

Retrieval workflows make this especially visible. A user asks a question. The system rewrites the query. It performs vector search. It may perform a keyword search. It reranks results. It compresses chunks. It passes context to the model. The model generates an answer. A verifier checks whether citations support the claims. If support is weak, retrieval may run again.

This can be the right design for a serious knowledge system. But it should not happen casually.

Each agent should have access only to the tools required for its role. Search should be used when the system lacks enough information, not as a reflex. Similar queries should be deduplicated. Results should be cached. Large tool outputs should be summarised or stored externally rather than replayed into every prompt.

A useful retrieval policy might look like this:

```
def should_retrieve(state):
    if state["answer_confidence"] >= 0.85:
        return False

    if state["cached_context_available"]:
        return False

    if state["remaining_budget"] < state["estimated_retrieval_cost"]:
        return False

    if state["retrieval_attempts"] >= 2:
        return False

    return True
```

This does not eliminate tools. It makes tool use intentional.

Retry loops turn small failures into expensive paths

Retries are necessary in production systems. Tools fail. APIs timeout. Schemas break. Retrieved context may be insufficient. Models sometimes produce outputs that do not pass validation.

The issue is not retrying. The issue is retrying without a budget.

A single malformed output can trigger a repair prompt. If the repair fails, the system may retry with a different instruction. If that fails, it may ask for a stronger model. If the tool result was incomplete, it may call the tool again. If the verifier rejects the answer, the planner may replan.

What began as one failed output can turn into a long execution path.

Reflection and self-critique loops create a similar pattern. They are useful when the task is difficult, and the extra reasoning improves quality. But if every request triggers reflection, the system pays for additional reasoning even when the answer was already good enough.

The right approach is to make retries and reflection conditional.

```
{
"max_model_calls": 5,
"max_tool_calls": 3,
"max_retries": 3,
"max_total_tokens": 50000,
"max_wall_clock_seconds": 10
}
```

When a workflow reaches its budget, the system should know how to degrade gracefully. It might return a partial answer with caveats, escalate to a human, ask for more information, or stop tool use and produce the best available answer.

A cost-aware agent knows when to stop.

Memory should reduce context, not add to it

Memory is often introduced as a solution to context limits. In principle, that makes sense. If the model cannot carry everything in the active prompt, store useful information externally and retrieve it when needed.

In practice, memory can either reduce cost or increase it.

It reduces cost when it replaces repeated context. It increases cost when it is added on top of everything else.

A poor memory design sends the full conversation history, a summary of the conversation, retrieved long-term memories, user profile data, previous tool logs, and external documents into the same prompt. This gives the appearance of a

memory-rich agent, but the cost profile is poor. The model now has more to read, not less.

A better design separates memory into tiers.

```
Active context:
  information required for the current step

Session memory:
  recent unresolved state

Long-term memory:
  durable facts and preferences

External storage:
  files, traces, reports, large outputs
```

The system should retrieve memory only when it is relevant. It should inject only the part that helps the current step. It should expire temporary facts. It should avoid storing transient reasoning as durable memory.

Memory should act like a filter. It should reduce what the model needs to see.

Observability is required, but it has its own cost

Production agent systems need observability. Without tracing, teams cannot understand why an agent selected a tool, where a bad output came from, which prompt version caused a regression, or why a request became expensive.

In multi-agent systems, observability is even more important because the failure may not be in the final answer. It may be in the route, the plan, the retrieval step, the tool result, the verifier, or the synthesis stage.

However, observability is not free.

A trace for a normal API request may include request IDs, timings, status codes, and service metadata. A trace for an agentic workflow may include prompts, completions, tool inputs, tool outputs, retrieved chunks, intermediate states, token

counts, latency metrics, model names, provider routes, retries, and evaluation labels.

That is a much heavier data footprint.

The answer is not to remove tracing. The answer is to make tracing proportional. During development, full-fidelity tracing is useful. For production traffic, high-risk paths may still need full traces. Routine low-risk paths may only need sampled traces and aggregate metrics. Failed requests should preserve more detail. Expensive outliers should be captured automatically.

Cost-aware observability asks:

> *Which traces are valuable enough to store in full?*

That question becomes important as traffic grows.

A cost-aware architecture

A cost-aware multi-agent architecture begins before the first model call.

It starts with a budget gate. The system estimates task complexity, user intent, risk level, available context, likely tool needs, and expected cost. Based on that estimate, it chooses the cheapest reliable execution path.

Simple requests should stay simple. Complex requests should get more reasoning. Risky requests should get verification. Unclear requests should trigger clarification before expensive orchestration.

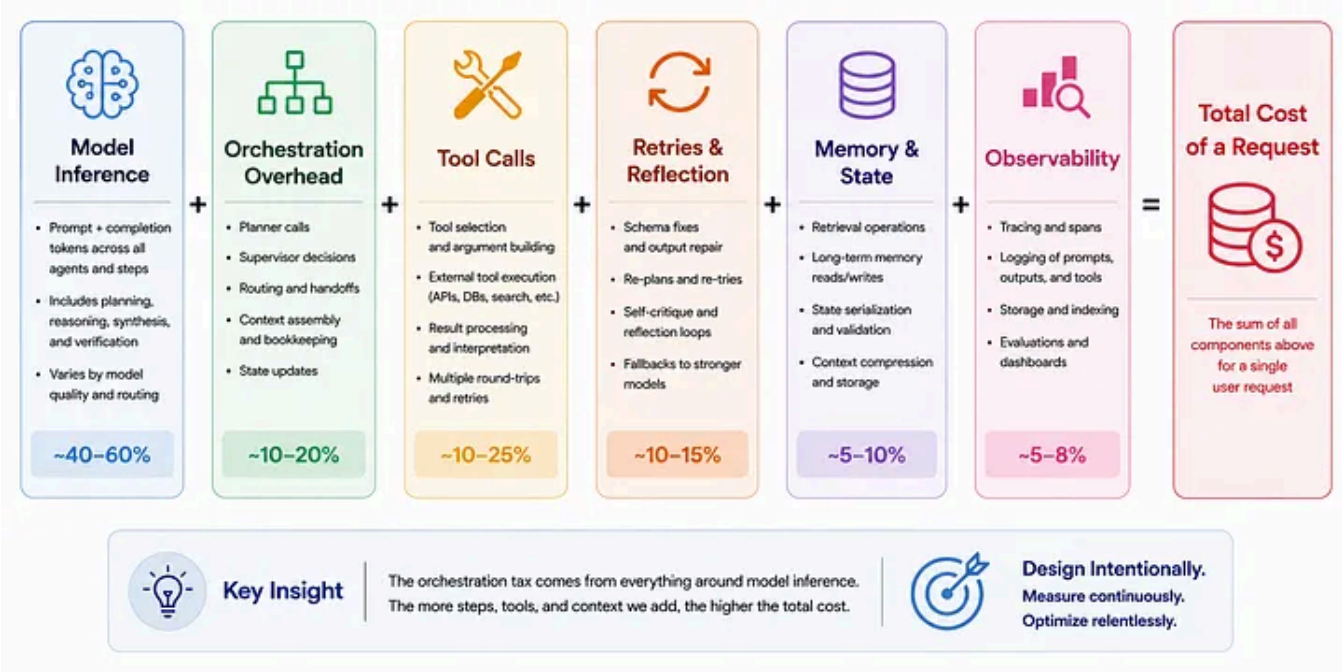


Figure 3. The Orchestration Tax Stack

A strong architecture usually includes routing, planning, scoped execution, gated tools, selective memory, verification, and monitoring.

The router decides whether the request needs a single-agent path, a tool path, a planner path, or an escalation path. Planning is used only when decomposition genuinely helps. Worker agents receive focused task packets rather than the full state. Tools and memory are activated only when they are expected to improve the result. Verification runs for high-risk outputs. Monitoring records cost, latency, model calls, tool calls, retries, and quality signals by workflow path.

The practical optimisation pattern

Cost optimisation works best when it is layered.

- Trim the state before every model call. Conversation history should not grow indefinitely. Older turns should be summarised, filtered, or removed unless they are still relevant.
- Use structured handoffs. Agents should communicate through compact payloads rather than raw transcripts.
- Route models by task difficulty. Small models should handle routine work. Stronger models should handle ambiguity, planning, and high-risk judgement.

- Restrict tool access. Each agent should only have the tools required for its role.
- Add hard budgets. A production system should track remaining token, call, time, and cost budgets during execution.
- Sample observability. Full traces are valuable, but not every production path needs full trace retention.

No single optimisation solves everything. The savings come from the combination.

Cost optimisation in multi-agent AI is not about making everything cheap. It is about making expensive steps earn their place.

A strong production system uses deterministic logic where exact checks are enough. It uses small models for routine work. It reserves stronger models for high-value reasoning. It passes scoped context instead of full transcripts. It gates tools. It bounds retries. It manages memory carefully. It observes cost at the level of agents, tools, workflow paths, and outcomes.

The systems that scale will not be the ones with the most agents. They will be the ones with the clearest control over when agents are needed, what they see, which tools they use, how long they can continue, and how much they are allowed to spend.

Before adding another agent, ask a harder question:

> ***Does this step improve the result enough to justify the additional cost, latency, and complexity?***

That question is where production-grade agentic architecture begins.

Generative Ai Tools | Genai | Cost Optimization | Multi Agent Systems | Generative Ai

Following

Published in Towards AI

138K followers · Last published 2 hours ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: https://email-course.towardsai.net/

Follow

Written by Deepanshu Gupta

9 followers · 11 following

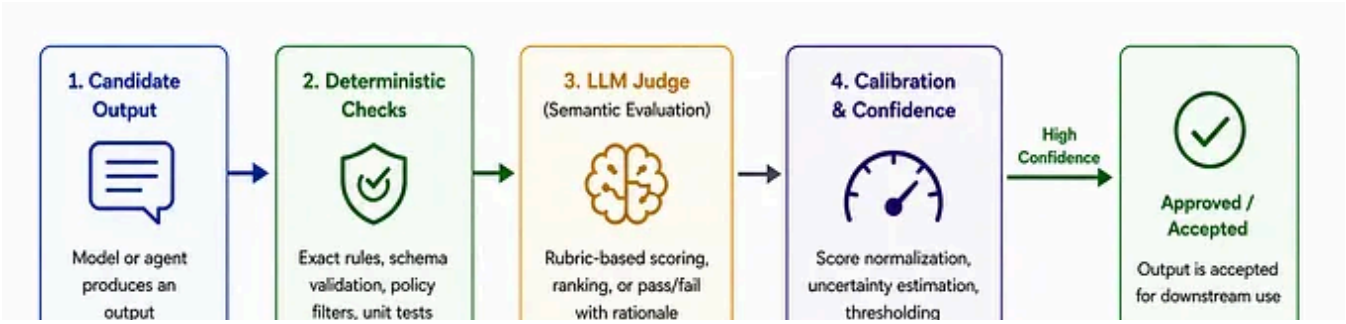
AI Engineer with experience building production-grade AI solutions. Exploring AI systems and the engineering challenges behind intelligent products.

No responses yet

Datachamp

What are your thoughts?

More from Deepanshu Gupta and Towards AI

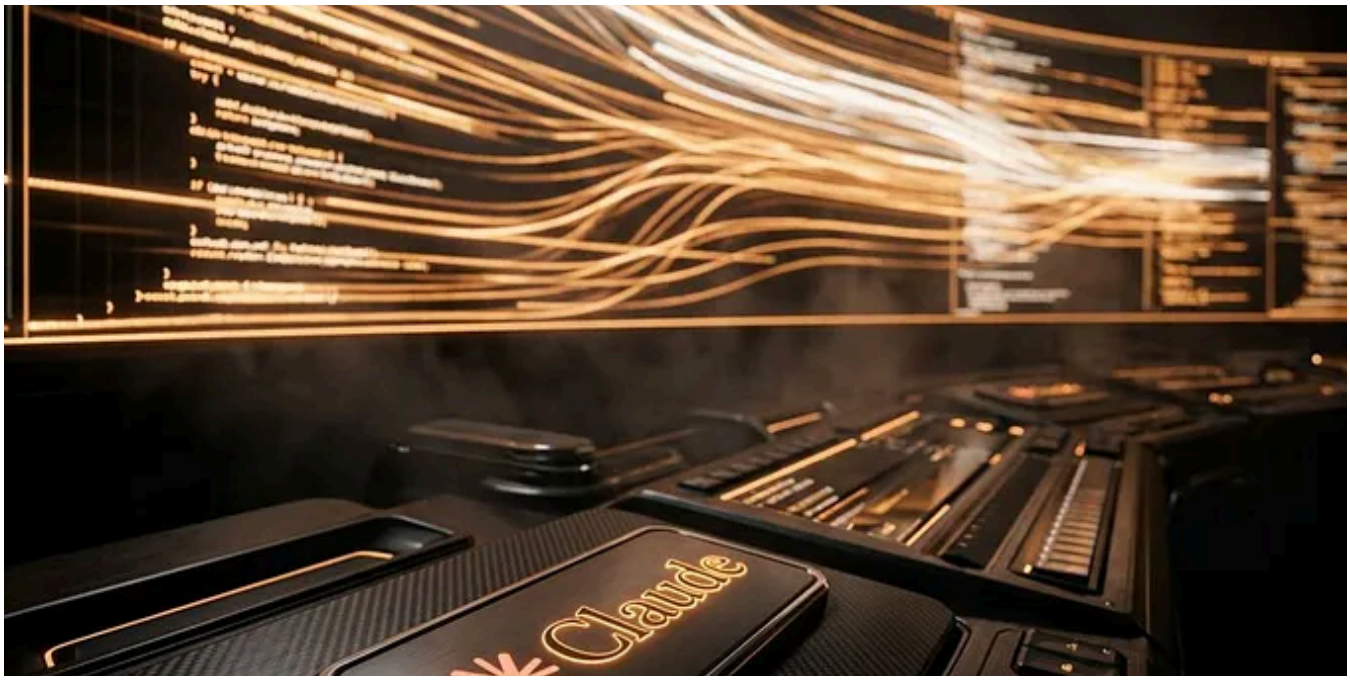


In Towards AI by Deepanshu Gupta · May 1

LLM-as-a-Judge : Designing Reliable AI Evaluators for Modern Agentic System

Why robust AI evaluation depends on more than stronger models.

1

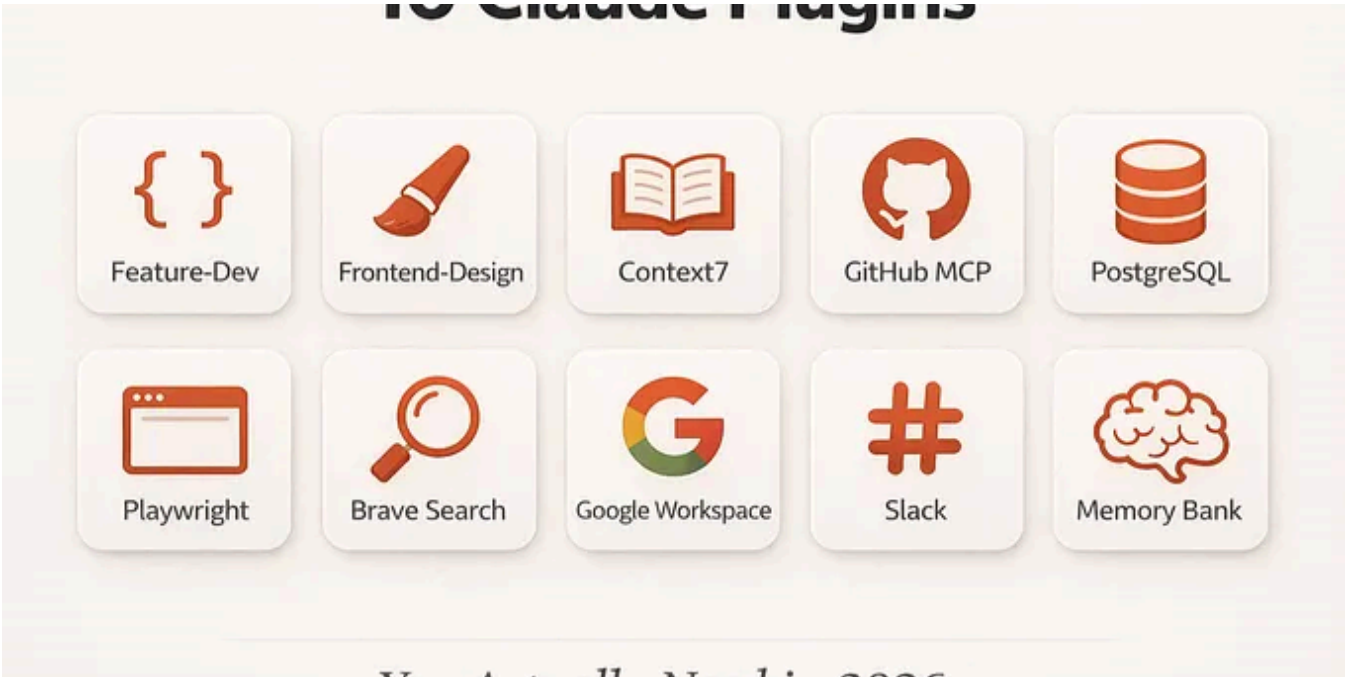


In Towards AI by allglenn · Apr 12

Becoming a top 1% Claude Code user: the complete playbook no one else is sharing

Most developers use Claude Code like a fancy autocomplete. The top 1% treat it like a programmable engineering team. This guide shows you...

1.3K 28

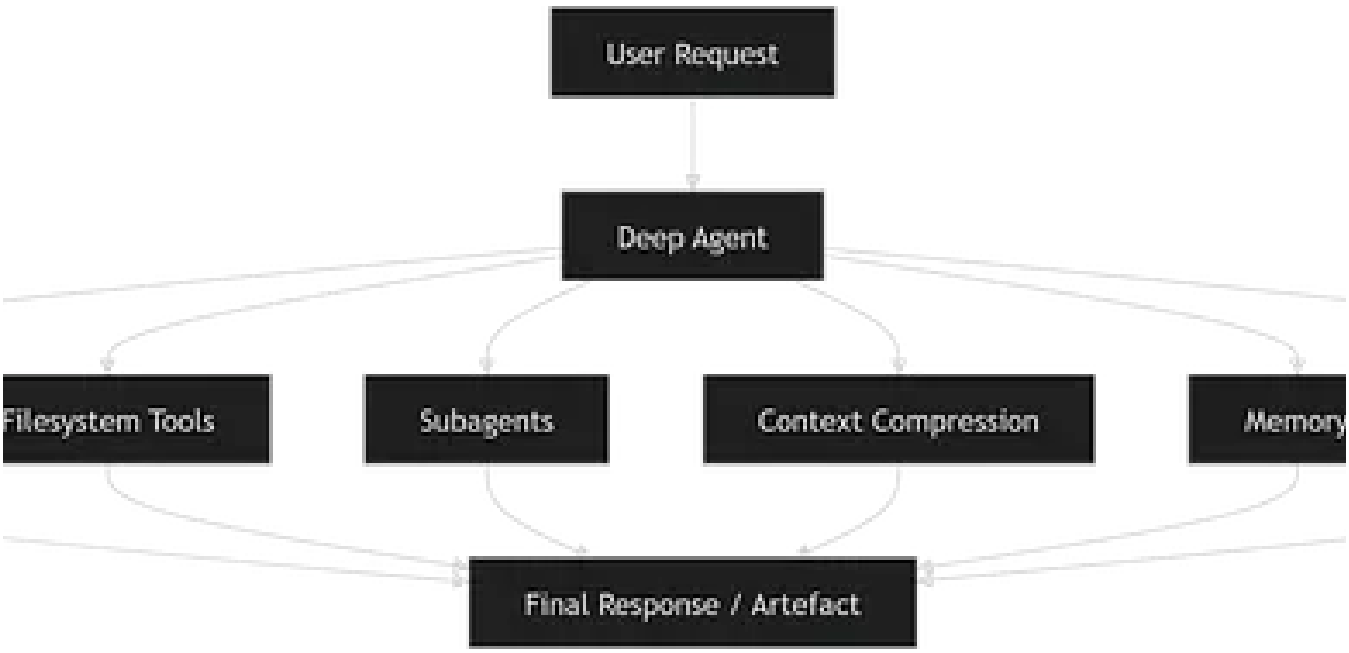


In Towards AI by Rohan Mistry · Mar 18

The 10 Claude Plugins You Actually Need in 2026 (And What They Are)

How to transform Claude from a chatbot into an AI that can code, browse, access your data, and automate your entire workflow

1.6K 21



In Towards AI by Deepanshu Gupta · Apr 18

DeepAgents: The Open-Source Framework for Building Long-Horizon AI Agents

Why LangChain's DeepAgents matters if you want agents that can actually finish complex work

See all from Deepanshu Gupta

See all from Towards AI